An Exclusion-Gated Method for Determining Internet Reachability of Vulnerabilities under the FedRAMP VDR/VER Rules

Matthew Venne
Chief Technology Officer, stackArmor

July 2026

Abstract

Few evaluation steps carry more weight under the FedRAMP Vulnerability Detection and Response (VDR) and Vulnerability Evaluation and Reporting (VER) rules—launched on 2026-06-24 as part of the FedRAMP Consolidated Rules for 2026, and applicable to both 20x and Rev5 offerings—than deciding whether a vulnerability is internet-reachable. The Internet-Reachable Vulnerability (IRV) determination (`FRD-IRV`, `VER-EVA-EIR`) decides which findings land on the tightest `VDR-TFR-PVR` clocks and which are Not Internet-reachable Vulnerabilities (NIRV). FedRAMP defines reachability as a property of *vulnerabilities*, not of network topology. A database on a private subnet, with no route to the internet, still hosts internet-reachable SQL-injection vulnerabilities if it processes data that came from the internet. Tools that only compute the internet-*accessibility* of resources therefore miss exactly the cases the definition exists to catch. This paper proposes a three-phase **Exclusion-Gated Reachability Method**. Phase A finds the assets the internet can reach directly, using four exclusion gates (routing, firewall source attribution, remote-access boundary authentication, and container routing) plus a process-level refinement. Phase B follows internet-derived data across the observed data-flow graph and prunes the assets that data can never reach. Phase C applies three per-vulnerability filters—execution evidence, trigger-surface matching, and trigger-mechanism class—to what remains. Every filter defaults to "reachable" unless hard evidence says otherwise, and every exclusion maps to a machine-readable Vulnerability Exploitability eXchange (VEX) justification. Finally, we take up the Web Application Firewall (WAF) question. FedRAMP explicitly permits treating perimeter-intercepted vulnerabilities as no longer internet-reachable; we recommend the stricter posture of keeping them classified IRV and attesting the mitigation via signed VEX—noting the `VDR-TFR-KEV` carve-out that Known Exploited Vulnerabilities must still be remediated on CISA BOD 26-04 due dates even when fully mitigated.

1 Introduction

On 2026-06-24, FedRAMP launched the Consolidated Rules for 2026, including the Vulnerability Detection and Response (VDR) and Vulnerability Evaluation and Reporting (VER) rule sets. The rules apply to both FedRAMP 20x and Rev5 offerings, and they replace scanner-centric continuous monitoring with an evaluation-driven model. Providers MUST now evaluate every detected vulnerability three ways: is it an Internet-Reachable Vulnerability (IRV, `VER-EVA-EIR`)? Is it a Likely Exploitable Vulnerability (LEV, `VER-EVA-ELX`)? And what Potential Agency Impact N-rating (PAIN, `VER-EVA-EPA`) could exploitation carry? Together those three answers pick the row

and column in the VDR-TFR-PVR timeframe matrix, and each must be reported per vulnerability under VER-RPT-VDT.

Getting reachability wrong is costly in both directions. Call too many internal flaws IRV, and the security team drowns in artificial emergencies on clocks the rules never meant to impose. Call a genuinely reachable flaw NIRV—FedRAMP's term for a Not Internet-reachable Vulnerability (FRD-IRV)—and an exploitable path stays open past its intended clock. VER-EVA-ELX adds a warning: evaluations made "recklessly or deliberately" wrong will damage a provider's FedRAMP Certification.

The hard part is that modern cloud architectures are layered and constantly changing. Blanket assumptions—every asset in a public subnet is reachable, every container in a GKE/EKS cluster is reachable—are too coarse. Manual, case-by-case declarations are unverifiable and will not survive a rigorous 3PAO or agency review. This paper proposes an **Exclusion-Gated Reachability Method**: every vulnerability is assumed internet-reachable *by default*, and is excluded only when hard, auditable evidence rules it out. The method runs in three phases, described in Section 4.

Most reachability tooling in the market today computes the internet-*accessibility* of resources and stops there. Section 2 explains why that is not the question FedRAMP asks—and why the wrong question produces false negatives on the very examples FedRAMP's own rule notes use as motivation: SQL injection and Log4Shell.

2 Reachability Attaches to Vulnerabilities, Not Resources

FedRAMP defines an Internet-Reachable Vulnerability (FRD-IRV) as:

> "A vulnerability in a machine-based information resource that might be exploited or otherwise triggered by a payload originating from a source on the public internet."

The definition's notes remove any ambiguity about topology: it explicitly "includes machine-based information resources that have no direct route to/from the internet but receive payloads or otherwise take action triggered by internet activity," while also limiting scope in the other direction—"internet-reachability applies only to the specific vulnerable machine-based information resources processing the payload."

The evaluation rule VER-EVA-EIR states the design intent directly: "FedRAMP focuses on internet-reachable (rather than internet-accessible) to ensure that any service that might receive a payload from the internet is prioritized if that service has a vulnerability that can be triggered by processing the data in the payload." Its notes supply two canonical examples. The first is SQL injection, "where an application stack behind a load balancer and firewall with no ability to route traffic to or from the internet can receive a payload indirectly from the internet that triggers the manipulation or compromise of data in a database that can only be accessed by an authorized connection from the application server on a private network." The second is Log4Shell, "where exploitation was possible via vulnerable internet-reachable resources deep in the application stack that were often not internet-accessible themselves."

These examples doom any method that treats reachability and accessibility as the same thing. A routing-and-firewall analysis will correctly conclude that the private-subnet database is not internet-accessible—and will then mark its vulnerabilities NIRV, which is exactly the false negative FRD-IRV exists to prevent. VER-EVA-EFA points the same direction: it lists *Reachability* ("How might a threat actor reach the vulnerability and how likely is that?") among the evaluation factors, alongside exploitability, prevalence, and privilege. A sound method must answer a question about where data flows, not where packets route:

Can data that originated on the public internet arrive—directly or through any chain of intermediaries—at the vulnerable code, in a form that could trigger the vulnerability?

Accessibility analysis remains necessary—it identifies the entry points where internet data comes ashore—but it is only the first phase of the determination, not the determination itself. This paper keeps the two words strictly apart. *Internet-accessible* describes an entry point: an asset that accepts traffic arriving directly from the public internet. *Internet-reachable* is FedRAMP's broader status: it applies to any vulnerability that internet-originated data can reach and trigger, whether its host is an entry point or sits downstream of one. Every internet-accessible asset hosts candidates for reachability; the reverse is nowhere close to true.

IN PLAIN TERMS

The question is not "can the internet connect to this server?" but "can data that started on the internet ever reach this flaw?" A database with no internet route still hosts internet-reachable injection flaws, because the attacker's input walks in through the public application and is handed to the database by your own tier-to-tier plumbing. Tools that stop at server exposure answer the wrong question — and go wrong on exactly the cases FedRAMP names (SQL injection, Log4Shell).

3 Scope Realism: What the IRV Determination Actually Controls

Before presenting the method, it helps to be clear about the stakes, because the IRV determination alone does not set the tight clock. The `VDR-TFR-PVR` matrix is indexed by *three* evaluations at once: the PAIN rating picks the row, and the combination of likely exploitability and internet reachability picks the column (LEV + IRV, LEV + NIRV, or NLEV). Table 1 reproduces the Class C matrix, likely the most common certification class.

PAIN Rating	LEV + IRV	LEV + NIRV	NLEV
PAIN-5	2 days	4 days	16 days
PAIN-4	4 days	8 days	64 days
PAIN-3	16 days	32 days	128 days
PAIN-2	48 days	128 days	192 days

Table 1: `VDR-TFR-PVR` mitigation/remediation timeframes for Class C certifications. Classes A and B share a more generous schedule on the same structure; Class D is tighter.

Three observations keep the reachability problem in proportion:

- **LEV gates the tight columns.** The IRV and NIRV columns only differ for vulnerabilities that are also likely exploitable, and `VER-EVA-ELX` is blunt that those are a minority: "the simple reality is that most traditional vulnerabilities discovered by scanners or during assessment are not likely to be exploitable." So reachability matters most for the LEV subset. Note, though, that reachability feeds the LEV call itself: `FRD-LEV` requires the vulnerability to be "reachable by a likely threat actor," and its floor note makes any vulnerability "that an automated unauthenticated system can exploit over the internet" automatically LEV.
- **The timeframes are should-level and are satisfied by mitigation.** `VDR-TFR-PVR` asks providers to "partially mitigate, fully mitigate, or remediate" *to a lower Potential Agency Impact N-rating* within the timeframe. A partial mitigation that provably lowers the PAIN rating

satisfies the clock. (FedRAMP's definition of mitigation is broader than the rule's wording suggests: `FRD-PMV` covers reductions in likelihood *or* PAIN—Section 8 returns to this.) That does not end the process—the finding continues on the new, longer clock, and each reduction is reported under `VER-RPT-VDT`—but the descent is bounded. It ends in remediation, full mitigation, PAIN-1 (which carries no timeframe), or formal acceptance: `VER-TFR-MAV` requires any vulnerability that will not be fully mitigated or remediated within 192 days of evaluation to be categorized as an accepted vulnerability. No single timeframe demands full remediation (with the KEV exception discussed in Section 8).

- **The goal is determinism, not deadline-shrinking.** This method does not exist to shrink the IRV list through aggressive interpretation. It exists to make each call deterministic, evidence-backed, and auditable, so the LEV + IRV set a provider reports under `VER-RPT-VDT` can be defended line by line—to a 3PAO, an agency, or FedRAMP itself.

IN PLAIN TERMS

Reachability is one of three dials, and rarely decisive alone: the tightest clocks bind only flaws that are *also* likely exploitable — a minority — and each clock is satisfied by demonstrably knocking the risk down a rung, not only by a full fix. So the goal here is not to argue findings off the urgent list; it is to make every reachability call one you can prove to an auditor.

4 Method Overview: Three Phases

The method evaluates the pair (v, a)—vulnerability v on asset a—through three phases, illustrated in Figure 1.

Default state: every vulnerability on every asset is assumed internet-reachable (IRV).

Phase A — Direct Exposure $E(a)$ *(asset-level; identifies entry points)*

Gate 1: IP and routing: no public IP / NAT / LB mapping and no internet-gateway route.

Gate 2: Firewall source population: all permitted sources enumerable and attributable to identified organizations.

Gate 3: Remote-access boundary authentication: a dedicated edge component (IAP, VPN—never the application's own login page) rejects unauthenticated traffic, its declared, assessor-validated role is remote access for the organization's own people and tooling (not customer authentication for the service), and the backend keeps its own independent login behind it.

Gate 4: Kubernetes/container routing: no public Gateway route, Ingress, NodePort, `hostNetwork`/`hostPort`, or LoadBalancer exposure.

Plus a process-level refinement (Gate 5): software that binds no exposed listening socket is not *directly* exposed. Phase A output feeds Phase B; **it excludes nothing from IRV scope by itself.**

↓

Phase B — Transitive Payload Exposure $T(a)$ *(asset-level; data-flow graph)*

Propagate taint from Phase A entry points across observed data flows (NetworkPolicies + flow telemetry). Assets with *no* flow path from any entry point are pruned: all their vulnerabilities are NIRV. Tainted assets proceed to Phase C.

↓

Phase C — Per-Vulnerability Filters *(vulnerability-level, on tainted assets)*

Filter 1: Execution evidence: vulnerable code not present in, or never loaded into, the execute path.

Filter 2: Trigger-surface matching: required protocol/format never delivered on any inbound tainted edge.

Filter 3: Trigger-mechanism class: peer-bound vulnerability classes on assets that are not directly exposed.

Final classification: vulnerabilities surviving all three phases are IRV.

Figure 1: The three-phase exclusion-gated method. Phase A computes direct exposure, Phase B prunes assets unreachable by internet-derived data, and Phase C filters individual vulnerabilities on payload-exposed assets.

The phases compose into a single decision rule. Let $E(a)$ denote that asset a is directly exposed (survives Phase A), $T(a)$ that a is transitively payload-exposed (Phase B), and let $\mathrm{exec}(v, a)$, $\mathrm{surface}(v, a)$, and $\mathrm{mech}(v, a)$ denote that Phase C filters 1–3 respectively *apply* (i.e., fail to exclude). Then:

$$\mathrm{IRV}(v, a) \;=\; \big[\, E(a) \;\vee\; T(a) \,\big] \;\wedge\; \mathrm{exec}(v, a) \;\wedge\; \mathrm{surface}(v, a) \;\wedge\; \mathrm{mech}(v, a) \tag{1}$$

Every term defaults to *true* unless hard evidence says otherwise. That default is the method's central safety property: missing telemetry, unknown trigger surfaces, or unclassifiable vulnerabilities can only make the IRV set *larger*, never quietly smaller.

IN PLAIN TERMS

Every flaw starts out presumed internet-reachable. It leaves that list only by passing hard-evidence checks: no internet-derived data ever reaches its host (Phases A–B), or the data that does arrive can never trigger this particular flaw (Phase C). Missing telemetry or an unrecognized bug class defaults to "still reachable" — so evidence gaps make the urgent list longer, never quietly shorter.

5 Phase A: Direct Exposure Gates

Phase A answers the classic accessibility question: which assets can the public internet deliver packets to directly? Its output is the set of *entry points*—the places where internet data first lands. To be explicit: **Phase A feeds Phase B; failing a Phase A gate does not, by itself, exclude anything from IRV scope.** The private-subnet database in FedRAMP's SQL-injection example fails Gate 1 and is still IRV for its injection flaws, because Phase B will find the data path through the application tier.

5.1 Gate 1: IP Address and Routing

An asset is directly exposed at the network layer only if two preconditions hold:

1. The asset holds a routable public IP address, or belongs to a NAT or load-balancer pool that maps public traffic to it.
2. The subnet's routing table has an active route to an internet gateway (`0.0.0.0/0` via an IGW or equivalent).

Assets failing either condition—private and isolated subnets, egress-only NAT configurations—are not entry points. They remain fully eligible for transitive payload exposure in Phase B.

5.2 Gate 2: Firewall Source Population and Attribution

Even with a public IP and an internet route, an asset whose inbound rules restrict sources to specific known networks is not exposed to the general internet. The tempting way to formalize that is a size rule—exclude whenever every permitted source prefix is `/20` or smaller—but any size rule fails in both directions. It can be gamed: one hundred separate `/24` rules each pass the per-rule test while together covering more address space than a `/16`. And it flags safe configurations: a single allowlisted partner `/16` fails the test even though it represents exactly one known organization.

The gate therefore looks at each listening port, takes the *union* of every source range permitted by any firewall rule, security group, or WAF-layer IP restriction that governs it, and asks who owns that space:

> **Exclusion criterion.** The asset/port is excluded from direct exposure if and only if the union of permitted sources is enumerable and every constituent range is attributable to an identified organization under an agreement with the provider—corporate networks, named partners, or specific customer egress ranges on record.

Size survives only as a red flag. A union larger than a `/20`-equivalent (4,096 addresses) SHOULD trigger a review of the attribution records, because allowlists that big tend to accumulate ranges nobody can explain. But size only decides when to look harder, never the outcome: a fully attributed partner `/16` passes; a hundred anonymous `/24`s do not.

IN PLAIN TERMS

An allowlist earns an exclusion by being *accounted for*, not by being small. If every permitted address range can be traced to a named organization you have a relationship with, the door is closed to the general internet; if parts of the allowlist are anonymous, it is not — no matter how narrow the individual ranges look.

5.3 Gate 3: Remote-Access Boundaries (Edge Authentication)

Many architectures put backends behind a boundary that checks identity at the edge: an identity-aware proxy (Google Identity-Aware Proxy, AWS Verified Access, ALB OIDC/Cognito authentication) or a client VPN. Both reject unauthenticated traffic before the backend ever processes it. The boundary must be exactly that—a component *separate from the application.* An application's own login page never qualifies: the login form, credential parsing, password reset, and session issuance are unauthenticated internet input processed by the application itself, so the pre-authentication attack surface *is* the application. Yet for true edge components, convention treats the two kinds very differently: backends behind a VPN are habitually called "internal," while backends behind an IAP are often scored as exposed. This gate applies one functional test to both:

> *Edge authentication qualifies for exclusion from direct exposure when it functions as* ***remote-access control*** *for the organization's own people and tooling—the clientless-VPN role, fronting admin consoles, operations tooling, and internal dashboards. It does not qualify when it functions as* ***customer authentication*** *for the service itself. The difference is the boundary's role, not the size of the crowd behind it.*

Rejecting unauthenticated traffic before the vulnerable resource sees it satisfies the interception principle in `VER-EVA-EIR`: "the simplest way to prevent exploitation of internet-reachable vulnerabilities is to intercept, inspect, filter, sanitize, reject, or otherwise deflect triggering payloads before they are processed by the vulnerable resource; once this prevention is in place the vulnerability should no longer be considered an internet-reachable vulnerability." For *unauthenticated* traffic, an enforcing IAP or VPN is exactly that prevention. The question that remains is simple: *who can still deliver data after logging in?* For a remote-access boundary, the answer is the organization's own people. For customer authentication, the answer is the service's internet user base. In verifiable form:

> **Exclusion criterion.** A vulnerability v on backend asset a may be excluded from direct exposure via edge authentication if and only if all of the following hold:
>
> (a) the boundary rejects unauthenticated traffic before v's vulnerable code processes any attacker-controlled data;
>
> (b) the boundary's role is remote access—it fronts the organization's own personnel and tooling, not the customer-facing service. The role is a recorded assertion: the operator declares it and the assessor validates it, the same way an assessor confirms what any other control is actually doing. It is not a head-count. A change-management system behind an IAP at a large provider may have a hundred thousand authorized users and is still remote access; a customer portal with three hundred accounts is still the service's front door; and
>
> (c) the backend verifiably validates the boundary's cryptographic identity assertion (e.g., validating the IAP-signed JWT on every request), so that header spoofing or misrouting around the boundary does not silently void the exclusion; and

(d) the backend authenticates its users *independently* of the boundary—a second login flow, the way a VPN user still signs into the console behind the tunnel. The boundary's signed assertion in (c) is bypass protection, not authentication. A backend that consumes the boundary's signed headers as its entire authentication story is one hijacked edge session or one compromised proxy away from granting authenticated access—a compromised IAP will cheerfully mint valid signed headers to the backend—so such a backend is treated as directly exposed.

Condition (b) is where the usual intuitions succeed and fail. "Behind the VPN, therefore internal" quietly relies on it—and it is usually right, because VPNs and identity-aware proxies overwhelmingly front an organization's own tooling. But when (a) holds and (b) fails—the login wall is the front door of the customer application—internet data still flows through authentication and into the backend. SQL injection behind the customer login is still an IRV, whatever brand of proxy enforces that login. In that setup, edge authentication is an *exploitability* input, not a reachability exclusion: it defeats the `FRD-LEV` floor ("any vulnerability that an automated unauthenticated system can exploit over the internet is a likely exploitable vulnerability"), which may support an NLEV call under `VER-EVA-ELX`, but it does not make the data path disappear.

IN PLAIN TERMS

An IAP or VPN in front of your *staff* tools is a real boundary: it exists to let your own people in, so what sits behind it is not internet-reachable — no matter how many staff you have. The same kind of login as the front door of your *customer* product excludes nothing — anyone on the internet can hold an account and send hostile input straight through it. What matters is what the door is *for*, not the door itself or the number of keys. Two more rules keep it honest: the door must be a separate component, not the app's own login page — a login page is the app parsing stranger input — and the app behind the door still needs its own login, just as tools behind a VPN do. One door is never enough: if the app trusts the proxy's stamp as the whole login, whoever beats the proxy owns the app. (Either way, the perimeter itself — the proxy or VPN concentrator — is internet-accessible.)

In practice, the failing case is rare. Almost every identity-aware proxy in a FedRAMP environment sits in front of staff tools—the qualifying role. Putting an edge login in front of the customer application itself is uncommon. So condition (b) rarely costs a real deployment anything. Its job is to close a loophole: without it, any login page would make everything behind it "not internet-reachable" on paper, including a SQL injection that fires after login. No assessor would accept that, and it is the first thing one would test. That is also why the role call belongs to people, not to automation. A dedicated boundary component that provably rejects unauthenticated traffic is treated as remote access by default; the operator declares any customer-facing exception in governed metadata, and the assessor confirms during assessment that the declarations match what the service actually does—determining whether an auth proxy is performing customer-facing authentication is the assessor's job, as is confirming that the backend's own authentication under condition (d) is a real second flow and not a pass-through of the boundary's headers. Cloud signals can flag a declaration for review (a GCP IAP policy granting `allUsers`, for example, is worth a question), but they refine the audit; they do not make the determination. The companion tooling specification covers the details.

No matter how the boundary is configured, the perimeter itself is internet-accessible. The VPN server, the load balancer, the proxy—anything that touches traffic before the login check—is exposed by definition. A login wall cannot protect the thing that *is* the login wall. The two technologies differ only in how much sits in front of that check. A VPN turns strangers away before a connection ever reaches anything behind it, so the VPN server is the only exposed piece. An identity-aware proxy decides later: the cloud load balancer has already accepted the connection

and parsed the request before the login check runs, so the load balancer and its parsing stack are exposed too. Behind the boundary, the exclusion works the same either way; what differs is which front-line components are internet-accessible.

5.4 Gate 4: Kubernetes and Container Routing

Containers need their own check, because a pod's exposure is decoupled from its node's. A pod is a direct-exposure entry point if and only if it is the active target of at least one of:

(a) a Gateway API route (`HTTPRoute`, `TCPRoute`, etc.) bound to a public-facing Gateway;
(b) an `Ingress` bound to a public load balancer;
(c) a `Service` of type `LoadBalancer` with a public address;
(d) a `NodePort` service on nodes reachable from the internet; or
(e) `hostNetwork: true` or a `hostPort` mapping on a node with public exposure—this includes DaemonSets, which frequently run with host networking on every node and are easy to overlook in route-centric inventories.

Pods exposed only through internal load balancers or cluster-internal Services are not entry points (but remain Phase B candidates). Two verifiability caveats:

- **NodePort cannot be decided from inside the cluster.** A NodePort opens the port on every node, but whether the internet can reach it depends on node public IPs and cloud firewall rules—information the Kubernetes API does not have. The gate MUST combine cluster state with the node-level Gate 1/Gate 2 results, or fall back to an explicit, recorded operator assertion. With neither, the NodePort is treated as exposed.
- **Init containers.** In an exposed pod, `initContainers` run once, finish before the main containers start, and listen on nothing afterward, so they are not directly exposed. The exception is `restartPolicy: Always`, which turns an init container into a long-running sidecar; those are treated as primary containers.

5.5 Gate 5: Process-to-Port Refinement (and Its Limit)

Not every package on an exposed host is itself exposed. Software MAY be excluded from *direct* exposure when it maps, deterministically, to running processes that hold no internet-exposed listening socket—`sshd` vulnerabilities on a public web server whose port 22 is locked to attributed sources under Gate 2, for example. The mapping (package manifest joined with socket-ownership telemetry) MUST be automated and reproducible per asset.

This gate has a hard limit, and it is the central lesson of Section 2. A queue worker on a private VM with *no listening socket at all*—a thumbnail generator, a log ingester, exactly the Log4Shell shape—is still IRV for its parser flaws if it parses internet-uploaded files. "Binds no exposed port" only excludes a vulnerability together with the Phase B condition "consumes no internet-tainted data." Gate 5 refines direct exposure; it is never a standalone NIRV call.

6 Phase B: Transitive Payload Exposure

Phase B implements the `FRD-IRV` note that reachability "includes machine-based information resources that have no direct route to/from the internet but receive payloads or otherwise take action triggered by internet activity." It works on the *data-flow graph*, not the routing graph. Nodes are assets (or workloads). A directed edge $a \to b$ exists when data observed leaving a arrives at b and is processed there—requests, messages, files, replication streams. An asset is **payload-exposed** (tainted), written $T(a)$, if any directed path leads to it from any Phase A entry point.

Perfect data-flow analysis is impossible at this scale, so the method deliberately over-counts, using two kinds of evidence:

- **Kubernetes estates:** the union of the flows the NetworkPolicies allow (the declared graph) and the flows actually seen by CNI or mesh telemetry, such as Cilium Hubble or Istio flow logs (the observed graph). The two kinds of evidence do different jobs. Observed flows only ever *add* edges the declared picture missed: seeing traffic proves a path exists, but not seeing traffic never proves one does not, because nothing stops a new flow from starting tomorrow. Only enforcement can rule paths out—policies that cover every workload, with default-deny behind them. Where NetworkPolicies are missing or default-allow, every path MUST be assumed open, and the permissive posture is itself a finding.

- **Non-Kubernetes estates:** the union of the flows that firewall and security-group rules allow (the declared graph) and the flows actually seen in VPC flow logs joined with host-level socket telemetry—which process held the socket, which peers it exchanged data with (the observed graph)—giving per-process flow edges rather than bare IP adjacency. The same division of labor applies: observed flows add edges the rules missed, and only deny-by-default firewall and security-group rules can rule paths out. Where the rules are default-allow or overly broad, every path MUST be assumed open, and the permissive posture is itself a finding.

Taint then spreads by simple repetition: start with the Phase A entry points, mark any asset that receives data from a marked asset, and repeat until nothing new gets marked. Because the flow evidence is collected continuously, the graph—and with it the IRV determination—rebuilds itself as the architecture changes, consistent with the automation posture of `VDR-CSO-ADT`.

Phase B is the method's one wholesale pruning step, and it is built to err on the safe side:

- **No permitted path means NIRV, wholesale.** If no path *can* reach asset a from any entry point—an isolated batch cluster behind default-deny policy, an air-gapped management enclave, a database whose only permitted client is an untainted internal tool—then no internet payload can arrive there, and *every* vulnerability on a is NIRV. The claim MUST rest on one of two things: enforcement that denies the paths (default-deny rules covering the asset, with the policy snapshots as evidence), or an explicit operator attestation that the flow map is complete and no path in it leads to the asset. Quiet telemetry alone is never sufficient: watching the flow logs for a month and seeing nothing arrive proves only that nothing arrived that month. A monthly batch job, a failover path, or a disaster-recovery runbook can produce its first-ever flow the day after the window closes.
- **A path is not a conviction.** Taint means internet-derived data reaches the asset in some form—not that every flaw on the asset can be triggered by it. Tainted assets go on to the per-vulnerability filters of Phase C.
- **Direction and age matter.** Edges point one way: an asset that only *pulls* data from a tainted source still receives that data, so it is tainted; an asset that only *pushes* data to one is not. And observed flows MUST carry a bounded observation window, so flows that no longer exist age out of the graph.

IN PLAIN TERMS

Phase B follows the data, not the network diagram. Wherever internet-originated data flows — requests, queued messages, uploaded files, replication — every system it touches stays in scope. A system drops out entirely, flaws and all, only when something actually rules the paths out: deny-by-default rules that cover it, or the operator's signed statement that the flow map is complete and no path leads there. "We watched for a month and saw nothing" is not enough on its own — quiet logs prove the past, not the future.

7 Phase C: Per-Vulnerability Filters on Payload-Exposed Assets

Phase C brings back precision on tainted assets, honoring the `FRD-IRV` note that "internet-reachability applies only to the specific vulnerable machine-based information resources processing the payload." Each filter is a deterministic exclusion with a defined evidence source and a matching VEX justification (per the companion paper, *A CycloneDX VEX Profile for FedRAMP Rev5 VDR/VER*). When evidence is missing or unclear, each filter defaults to "applies"—the vulnerability stays IRV.

IN PLAIN TERMS

Phase B only says internet data reaches the machine. Phase C asks, for each individual flaw on that machine: could that data actually set *this* flaw off? Three checks. Does the vulnerable code ever run at all (Filter 1)? Does the data that arrives even come in the format that triggers this flaw (Filter 2)? And is this the kind of bug that travels in data, or one that needs a direct connection from the attacker (Filter 3)? Each check needs solid evidence to cross a flaw off; without it, the flaw stays on the internet-reachable list.

7.1 Filter 1: Execution Evidence

The strongest and most general filter asks a blunt question: can the vulnerable code run at all? Two tiers of evidence:

- **The package never runs:** it is installed (so scanners flag it), but no process from it has run during the observation window—build leftovers, disabled agents, unused OS components. VEX justification: `code_not_present`.
- **The vulnerable function never runs:** the library loads, but runtime telemetry (eBPF-based library and symbol tracing, for example) shows the vulnerable function is never called. VEX justification: `code_not_reachable`.

(The corresponding CISA status-justification labels, under the companion profile's optional crosswalk, are `vulnerable_code_not_present` and `vulnerable_code_not_in_execute_path` respectively.) This filter works for every class of flaw—injection, parser bugs, protocol bugs alike—which makes it the highest-yield filter to automate. It also generalizes Gate 5: Gate 5 only removes *direct* exposure for processes that listen on nothing, while execution evidence rules out triggering entirely, no matter how tainted data arrives.

7.2 Filter 2: Trigger-Surface Matching

Every vulnerability needs its trigger delivered in a specific form—a protocol, a file format, an API endpoint. A flaw in a PDF parser cannot be set off by a JPEG; a flaw in Windows file sharing cannot be set off by database traffic. That required form is the vulnerability's *trigger surface*. Phase B already knows more than "these two machines talk": the flow telemetry records *what* is actually delivered on each edge (protocol, port, and often the application protocol, from mesh or eBPF-level classification). Filter 2 checks the two against each other:

> **Exclusion criterion.** Vulnerability v on tainted asset a is excluded if the trigger surface required by v has an empty intersection with the set of data formats and protocols actually delivered on a's inbound tainted edges.

EternalBlue—the SMB flaw behind WannaCry—makes the exclusion concrete. A scanner flags it on an interior Windows server, and Phase B confirms the server is tainted: internet-derived data really does reach it. But the flow telemetry shows that the only thing arriving on its tainted edges

is PostgreSQL database traffic from the application tier. EternalBlue fires only when the machine *parses SMB messages*, and no internet-derived data ever arrives as SMB. The vulnerable code never hears from the internet in the only language it listens to, so the finding is excludable—and the flow records that show it are the audit evidence. The exclusion polices itself in both directions. It rests on continuously observed flows: if someone later mounts a file share on that server and an SMB edge appears, the exclusion collapses on the next derivation. And it is narrow by construction: the same server's flaws in *PostgreSQL* parsing stay IRV, and an upload service that accepts arbitrary files excludes nothing, because "arbitrary files" matches every parser's trigger surface.

The CVE-to-surface mapping comes from enrichment data (affected component, CPE, vendor advisory). Where the required surface cannot be determined, **the filter assumes any surface could trigger the flaw, and the vulnerability stays IRV**. That default is the safety property again: incomplete knowledge widens the IRV set instead of narrowing it.

7.3 Filter 3: Trigger-Mechanism Class

The last filter sorts vulnerability classes by *how the trigger arrives*, keyed on CWE (available from NVD enrichment of CVE records):

- **Content-triggered** flaws are set off by *what the data says*, so they travel wherever the data travels. The canonical example is SQL injection (CWE-89): the attacker types a malicious string into a public search box, the web application passes that string to the internal database, and the database executes it. The attacker never connects to the database—their *text* does the work, and text survives every hop. Log4Shell worked the same way (CWE-917 expression-language injection): a booby-trapped string, logged by some service deep in the stack, triggered code execution on machines the attacker could not even name, let alone connect to. The same logic covers OS command injection (CWE-78), deserialization of untrusted data (CWE-502), memory corruption in file and format parsers (CWE-119 buffer overflow, CWE-787 out-of-bounds write—the malicious PDF or image that exploits whatever eventually opens it), XML external entity injection (CWE-611), server-side request forgery (CWE-918), and path traversal (CWE-22). **Content-triggered vulnerabilities stay IRV wherever tainted data flows, and can never be excluded by class** (Filters 1 and 2 may still exclude them). This is exactly FedRAMP's SQL-injection and Log4Shell concern.

- **Peer-bound** flaws are set off by *holding the connection*—and a connection, unlike data, cannot be forwarded. Heartbleed is the canonical example: to exploit it, the attacker's own machine had to open a TLS session with the vulnerable server and send malformed heartbeat messages over that session. regreSSHion (CVE-2024-6387) is another: the attacker must connect directly to the SSH port and race its login timer. The class covers flaws in the mechanics of the connection itself: the TLS or protocol handshake that opens it, the service's login check (CWE-287 improper authentication and CWE-306 missing authentication for a critical function, on the listening service itself), and connection-flood denial of service. The exclusion is justifiable for a physical reason, not a judgment call: when the web tier needs something from an interior server, it does not hand the attacker its connection. It opens a *new* connection of its own—its own handshake, its own credentials—and the attacker's data rides inside only as cargo. The attacker never holds a connection to the interior machine, so a flaw that lives in the connection mechanics has nothing to fire through. **Peer-bound vulnerabilities are therefore excludable on assets that are not directly exposed** ($\neg E(a)$): the only machines that can connect to an interior listener are the interior systems Phase B identified, and none of them is an internet

source. The exclusion never applies to machines the internet *can* connect to—Heartbleed on the public load balancer is directly exposed and stays IRV.

Two cautions keep this filter honest. First, stored XSS is content-triggered: the script arrives from the internet and is faithfully forwarded through the stack, even though it finally fires in a browser. "XSS is client-side, therefore excluded" is wrong. Second, not all denial of service is peer-bound: ReDoS and decompression bombs are triggered by the content of the data and follow it wherever it flows. A blanket exclusion of "XSS and DoS" is exactly the reckless evaluation `VER-EVA-ELX` warns against. The sound unit of exclusion is the combination in Equation 1—class, position, and taint together—and any vulnerability whose CWE is missing or fits neither group defaults to content-triggered and stays IRV.

8 Perimeter Mitigation: Permitted Downgrade, Recommended Attestation

One design question remains: what should happen to a vulnerability's IRV status when a Web Application Firewall rule—or a comparable payload-inspecting control, such as a virtual patch or an OWASP core-rule-set deployment—verifiably blocks the traffic that triggers it? There are only two choices: *reclassify* the finding NIRV, or *keep it IRV* and publish a signed Vulnerability Exploitability eXchange (VEX) attestation that it is mitigated at the perimeter. The conclusion of this section, up front: **FedRAMP permits the reclassification; we recommend the attestation.**

IN PLAIN TERMS

FedRAMP does allow you to stop counting a flaw as internet-reachable once a WAF rule verifiably blocks the payloads that trigger it. We recommend not taking the discount: WAF rules are bypassed, tuned, and toggled off far more easily than networks are re-plumbed. Keep the finding classified reachable and attach a signed, machine-readable note that it is mitigated at the perimeter — your dashboards keep showing the true residual risk, and the auditor gets a stronger artifact. One thing no WAF and no reclassification can move: known-exploited (KEV) flaws must still be *fixed* by CISA's due dates.

FedRAMP permits the downgrade. It is tempting to assume that reclassifying a WAF-shielded vulnerability as NIRV would itself draw audit findings. The launched rules say otherwise. The `VER-EVA-EIR` interception note quoted in Section 5.3 states that once payload interception "is in place the vulnerability should no longer be considered an internet-reachable vulnerability," and a WAF rule that verifiably intercepts the triggering payloads before the vulnerable resource processes them *is* such a prevention. **FedRAMP explicitly permits the downgrade.** Note that the note's verb list—"intercept, inspect, filter, sanitize, reject, or otherwise deflect"—is deliberately generic: the WAF is the canonical member of the class, not its boundary. Any control that verifiably stops the triggering payload *before the vulnerable resource processes it* qualifies on the same terms—an API gateway enforcing a request schema, upstream input sanitization, content disarm and reconstruction on uploaded files, a mail filter stripping the attachment type. Everything in this section applies to that whole class. The rules also lower the stakes: FedRAMP formally separates mitigation from remediation (`VDR-CSO-RES`: "a fully mitigated vulnerability will still exist (with negligible risk) until it has been remediated"; see `FRD-PMV`, `FRD-FMV`), and the `VDR-TFR-PVR` timeframes are satisfied by mitigation to a lower PAIN rating. A WAF virtual patch is a legitimate partial or full mitigation on its own, with or without any reclassification.

How an interception satisfies the clock. There is a drafting gap here worth closing, because it will be argued in assessments. `VDR-TFR-PVR`'s satisfaction language speaks of mitigating "to a lower Potential Agency Impact N-rating"—a *row* move—while a reachability downgrade moves the *column* and touches no PAIN rating at all. Read hyper-literally, the WAF that FedRAMP itself calls a mitigation could never satisfy the timeframe. The definitions close the gap: `FRD-PMV` defines a partially mitigated vulnerability as one whose "likelihood or Potential Agency Impact N-rating has been reduced," and `FRD-FMV` extends the same pair "until either are negligible." Interception is a *likelihood* reduction. A rule that verifiably blocks every variant of the triggering payload drives the likelihood of internet exploitation to negligible, making the finding *fully mitigated* under `FRD-FMV`—and a fully mitigated vulnerability leaves the timeframe treadmill for routine operations under `VDR-TFR-RMN` (KEVs excepted, below). Anything short of that is a partial mitigation on the likelihood axis: the finding is re-evaluated, lands in the NIRV (or NLEV) column, and continues on that longer clock, with the change reported under `VER-RPT-VDT`. The same logic is what lets *any* attested control in this section count as mitigation at all: the lever is the likelihood half of FedRAMP's own mitigation definition, not a PAIN reduction.

We recommend the attestation anyway. The reasons are practical, not regulatory:

- **WAF exclusions are fragile; topology exclusions are not.** Log4Shell showed why in real time. Within days of the first WAF rules blocking `${jndi:ldap://...}`, attackers published obfuscated variants—nested lookups like `${${lower:j}ndi:...}`—that sailed straight past the patterns, and every WAF vendor spent December 2021 chasing them. That is the difference between the two exclusions in one incident: a WAF exclusion is a bet that a signature matches every variant of the exploit an adversary can invent; a Phase B exclusion—the topology kind, granted only when deny-by-default rules or an attested-complete flow map show no path from any internet entry point to the asset (Section 6)—is a bet that an attacker cannot send data over a connection that does not exist. Nobody obfuscates their way through a missing path. The WAF bet also loses quietly to your own operations—a rule dropped to "detection-only" during an incident, a CDN migration, an emergency toggle each silently voids the downgrade—so reclassifying on WAF evidence means re-validating the call on every perimeter change. Easy to promise, hard to actually do.

- **When the WAF fails, the attested record is still true; the reclassified record is not.** Suppose the WAF is bypassed, misrouted, or toggled off. In reality both postures are now equally exposed—the difference is what the books say. The reclassified finding still reads NIRV on a slow clock, and that statement became false the moment the control failed, with nothing in the vulnerability record to say so and nothing prompting a re-evaluation. The attested finding still reads IRV—which is still true—and its VEX statement names the exact control that just failed, so the record degrades to what it should: internet-reachable, mitigation in doubt, tight clock. That is the case for attestation in one sentence: the reachability classification never depends on the minute-to-minute health of a perimeter device; only the mitigation claim does, and that claim names precisely what to watch.

- **Attestation is better audit evidence and keeps the risk visible.** Keeping the vulnerability IRV with a signed VEX record—`state: not_affected`, `justification: protected_at_perimeter`, per the companion CycloneDX profile—hands the 3PAO a signed, machine-readable assertion naming the exact protecting control, while the provider's own dashboards keep showing the true residual risk. (A VEX document is a signed statement by

the security team, not a legal instrument; its value is attribution and machine-readability, not liability transfer.)

- **Distribution automates cleanly.** VEX attestations publish to container registries as OCI artifacts next to the images they cover, or to object storage; scanning tools fetch them at scan time and apply the mitigation status without touching the reachability computation itself.

Known Exploited Vulnerabilities bend for neither approach. `VDR-TFR-KEV` says Known Exploited Vulnerabilities SHOULD be *remediated* by the due dates in the CISA KEV catalog, "even if the vulnerability has been fully mitigated," per CISA BOD 26-04. Log4Shell—this paper's running example—is a KEV. No WAF virtual patch extends a BOD 26-04 date, and neither does an NIRV reclassification. Providers MUST track the KEV clock separately from both the reachability call and the mitigation status.

No other control changes the reachability answer. The same question will be asked about the rest of the security toolbox: does a secure-configuration baseline, an EDR agent, or runtime protection make a finding NIRV the way a WAF rule can? No. Reachability changes only when internet-derived data can no longer arrive at the vulnerable resource—by removing the data path (Phase B, Gate 3) or by intercepting the triggering payload before the vulnerable resource processes it (this section's class of controls). The other controls are valuable, but they move different levers, and claiming NIRV on their evidence is a category error an assessor will catch:

- **Configuration that disables the vulnerable feature** (Log4Shell's `formatMsgNoLookups` flag), a removed package, code that never executes (Filter 1): the flaw cannot fire at all. That is a *disposition*—`not_affected` with `requires_configuration`, `code_not_present`, or `code_not_reachable`—and the finding leaves the active queue with its PAIN retained, per the companion method memo. The payload still arrives; it just has nothing to set off.
- **Enforcing runtime protection** (RASP, a seccomp or AppArmor profile that denies the system calls the exploit needs, or an EDR in *blocking* mode): the WAF's sibling, and treated almost identically. Keep the finding IRV, attest `protected_at_runtime`, and take the mitigation credit through the likelihood clause above; where the block verifiably contains what exploitation can accomplish—killing the process before persistence, denying the second-stage download—it is also cited evidence for lowering the Modified C/I/A metrics, and with them the PAIN rating, on the companion memo's ladder. The one thing it does not earn is the reachability downgrade, because it acts on the wrong side of the line: an interception stops the payload before the vulnerable code processes it, while a runtime control fires when exploitation is already underway—and for flaws whose damage completes during processing itself (Heartbleed's leak is already in the response, a ReDoS lockup has already happened), there is no second act left to block. Two cautions. The claim must be per-vulnerability: evidence that the blocking capability covers *this* CVE's exploitation technique, not the fleet-wide fact that an agent is installed. And an EDR in detect-and-alert mode earns none of this: noticing exploitation is response evidence, not prevention, and it changes no classification, no clock, and no disposition.
- **Hardening that shrinks the blast radius** (non-root users, read-only filesystems, egress denial, scoped credentials, compiler and OS protections): the flaw still fires, but it does less. That is the *impact* lever—the companion method memo's mitigation ladder lowers the Modified C/I/A metrics on cited evidence (`protected_by_mitigating_control`, `protected_by_compiler`)—and the finding descends to a lower PAIN rating on a longer clock.

The discipline is uniform: every control-based claim is attested in signed VEX, cites its evidence, and is re-evaluated the moment the control changes.

9 Summary and Conclusion

The FedRAMP VDR/VER rules ask a sharper question than the industry is used to answering: not "which resources are internet-accessible?" but "which *vulnerabilities* can a payload from the public internet trigger?" (`FRD-IRV`, `VER-EVA-EIR`). The Exclusion-Gated Reachability Method answers it in three deterministic phases. Phase A's gates—routing, attributed firewall sources, remote-access boundaries, container routing, and the process-level refinement—find where internet data enters. Phase B follows that data across the observed flow graph and prunes, wholesale and with flow logs as evidence, the assets it can never reach. Phase C excludes individual vulnerabilities on the remaining assets through execution evidence, trigger-surface matching, and the content-triggered/peer-bound split. Every filter defaults toward IRV, so evidence gaps show up as conservatism, never as silent false negatives—and every exclusion carries machine-verifiable evidence and a VEX justification a 3PAO or agency can check.

The method's purpose is proportionality with defensibility. Most detected vulnerabilities are not likely exploitable (`VER-EVA-ELX`), the tight `VDR-TFR-PVR` clocks bind only the LEV + IRV intersection, and partial mitigation legitimately satisfies the timeframes. A provider does not need to shrink its IRV set by declaration; it needs to defend the set it reports. Where a WAF virtual patch intercepts the triggering payloads, FedRAMP permits the reachability downgrade—but the stricter, more durable posture is to keep the vulnerability IRV and attest the mitigation with signed CycloneDX VEX, while still remediating Known Exploited Vulnerabilities on CISA BOD 26-04 due dates per `VDR-TFR-KEV`. Run the three phases as continuous automation, and the reachability determination rebuilds itself as the architecture changes—and stands up, line by line, to the scrutiny the new rules invite.

Glossary of Abbreviations

`3PAO`	Third-Party Assessment Organization
`ALB`	Application Load Balancer (AWS)
`BOD`	Binding Operational Directive (CISA)
`CDN`	Content Delivery Network
`CISA`	Cybersecurity and Infrastructure Security Agency
`CNI`	Container Network Interface
`CPE`	Common Platform Enumeration
`CVE`	Common Vulnerabilities and Exposures (vulnerability identifier)
`CWE`	Common Weakness Enumeration (vulnerability class taxonomy)
`DoS`	Denial of Service
`eBPF`	extended Berkeley Packet Filter (kernel-level runtime telemetry)
`EDR`	Endpoint Detection and Response
`EKS`	Amazon Elastic Kubernetes Service
`FRD`	FedRAMP Definition (rule-ID prefix, e.g. `FRD-IRV`)
`GKE`	Google Kubernetes Engine
`IAM`	Identity and Access Management
`IAP`	Identity-Aware Proxy
`IGW`	Internet Gateway
`IRV`	Internet-Reachable Vulnerability
`JWT`	JSON Web Token

KEV	Known Exploited Vulnerability (CISA catalog)
LB	Load Balancer
LEV	Likely Exploitable Vulnerability
MFA	Multi-Factor Authentication
NAT	Network Address Translation
NIRV	Not Internet-reachable Vulnerability
NLEV	Not Likely Exploitable Vulnerability
NVD	National Vulnerability Database
OCI	Open Container Initiative (registry artifact format)
OIDC	OpenID Connect
OWASP	Open Worldwide Application Security Project
PAIN	Potential Agency Impact N-rating (N1–N5)
RASP	Runtime Application Self-Protection
ReDoS	Regular-expression Denial of Service
SMB	Server Message Block (Windows file-sharing protocol)
SQL	Structured Query Language
SSH	Secure Shell
SSRF	Server-Side Request Forgery
TLS	Transport Layer Security
VDR	Vulnerability Detection and Response (FedRAMP rule set)
VER	Vulnerability Evaluation and Reporting (FedRAMP rule set)
VEX	Vulnerability Exploitability eXchange
VPC	Virtual Private Cloud
VPN	Virtual Private Network
WAF	Web Application Firewall
XSS	Cross-Site Scripting
XXE	XML External Entity (injection)